

Music Similarity Finder: An Interactive System for Audio Feature-Based Music Similarity and Visualization

Enis Herac

University of Sarajevo

Faculty of Electrical Engineering

Sarajevo, Bosnia and Herzegovina

cherac1@etf.unsa.ba

enisherac7@hotmail.com

Abstract— This paper presents Music Similarity Finder, a Streamlit-powered application for exploring music through audio feature analysis and radar chart visualization. The system extracts rhythmic, spectral, and timbral features from songs using Librosa, computes similarity metrics, and offers interactive recommendations based on user preferences for energy, danceability, and tempo. The project aims to make music discovery more interpretable by showing how songs relate in feature space, allowing users to understand why certain tracks are recommended. Results show that feature-based similarity offers intuitive and controllable exploration, demonstrating potential for transparent, user-centered music recommendation systems.

Keywords— *Music similarity, audio feature extraction, Librosa, Streamlit, radar chart visualization, content-based recommendation, music information retrieval (MIR), Euclidean distance, user preference modeling*

I. INTRODUCTION

In the era of AI-driven music platforms, music recommendation systems play a central role in how users discover and engage with songs. These systems analyze vast music catalogs to suggest tracks that match a listener's tastes, moods or listening history. Traditional recommendation systems generally fall into two categories: collaborative filtering and content-based filtering. Collaborative filtering relies on user behavior patterns such as playlists, listening history, and ratings to find similarities between users and predict preferences. Content-based filtering, on the other hand, analyzes the intrinsic properties of songs-like tempo, rhythm, and timbre to recommend similar tracks based on audio features. While these systems power popular platforms such as Spotify, Apple Music, and YouTube Music, they often operate as "black boxes," providing little insight into why a particular song is recommended. Music Similarity Finder addresses this limitation by offering a transparent, feature-based approach to music discovery. By extracting measurable audio attributes such as tempo, spectral centroid, and zero-crossing rate, the system visualizes each song's "Musical DNA" using radar charts. This interpretable interface allows users not only to explore similarities between tracks but also to influence recommendations in real time by adjusting preferences for energy, danceability, and tempo. Unlike traditional systems that rely solely on passive listening behavior, where users must play numerous similar tracks for the algorithm to

gradually refine its suggestions, Music Similarity Finder enables active control and instant feedback. By exposing the underlying audio features and allowing direct adjustment of preference weights, users can fine-tune the system's understanding of their tastes on demand. This transforms recommendation from a trial-and-error process into an interactive exploration, empowering listeners to discover music that genuinely aligns with their desired mood, tempo, or energy level.

II. LITERATURE REVIEW

The basis for contemporary music similarity analysis systems is in the realm of the so-called Music Information Retrieval and encompasses the study of how musically relevant information might be algorithmically extracted, modeled, and interpreted. An early study by Tzanetakis and Cook [1] showed the groundwork for content-based classification of music, relying upon low-level features describing the spectral centroid, zero-crossings, and the Mel-Frequency Cepstral Coefficients. The importance of such features relied upon the amount to which they correlate with human notions of brightness, energy, and timbre, as explicitly utilized for song descriptions in "Music Similarity Finder".

The calculation of MFCC coefficients, which play an integral part in processing both speech and music, has its roots in the work of Davis and Mermelstein [2], who proposed MFCC coefficients as an efficient way to represent the perceptual features of sound spectra.

This musical application was later adapted to music by Logan [3], who demonstrated how MFCC could be effectively used for music modeling and genre classification. These works were used for the "Musical DNA" feature.

The implementation backbone of the project is built upon Librosa, an open-source Python audio signal processing library [4]. Librosa provides the core algorithms used in Music Similarity Finder for computing tempo, spectral features, and MFCCs. It also ensures that efficient interaction can be achieved between it and the corresponding Python machine learning library, Scikit-learn, designed by Pedregosa et al. [5], which offers standardization and calculation functions of distance. Music Similarity Finder uses StandardScaler for feature normalization and Euclidean distance as its primary similarity measure.

Alternative measures of similarity have been explored extensively in MIR literature. Pampalk [6] surveyed various computational models of music similarity, highlighting both the potential and limitations of simple distance-based metrics.

The conceptual framework for user-adjusted preference weighting within the app is loosely inspired by Rocchio's [7] relevance feedback model in information retrieval. Rocchio's method allows users to iteratively refine query results by expressing positive or negative feedback, thereby guiding the retrieval process. In a similar spirit, Music Similarity Finder lets users bias the similarity computation toward features they prefer most-energy, danceability, or tempo-enabling personalized and controllable discovery. This introduces a dynamic, human-in-the-loop adaptation mechanism that most static recommendation systems lack.

In terms of interaction design, the foundation of visualization and interaction for the project is provided by Streamlit [8] and Plotly [9]. Streamlit helps create a dynamic and browser-based UI where the viewers can adjust settings in a real-time manner, and Plotly helps create a radar chart that enables viewers to understand the vector of features in an interpretable way.

In summary, these works together set the theoretical and practical foundations of Music Similarity Finder. From feature extraction ([1]–[4]) to distance modeling ([5], [6]) and interactive visualization ([8], [9]), the project integrates proven techniques from MIR and machine learning into an interpretable, user-centered application for music discovery.

III. IMPLEMENTATION AND CASE STUDY

A. Dataset Composition

The curated dataset of the project consists of 50 Suno AI-generated songs across five different genres: Pop, Hip Hop, Rock, Country, and Jazz. The dataset was created as a means of providing a controlled, copyright-free environment for testing music similarity algorithms without the various legal issues that come with commercial music. The metadata structure can be found in data/metadata.csv and keeps three important features for each track: filename, human-friendly title, and genre classification. This organizational approach enables both technical analysis (via filename references) and user-friendly presentation (through title display). The stratification on genre means that similarity algorithms will be tested across diverse music and not biased to any one style.

B. Use Case Scenarios

The system supports several practical use cases:

- Genre exploration: Users can discover how similarity transcends traditional genre boundaries
- Feature-based discovery: Finding songs with specific audio characteristics regardless of genre
- Preference experimentation: Testing how different preference adjustments affect recommendations
- Educational demonstration: Visualizing how audio features translate to perceptual similarity

IV. IMPLEMENTATION DETAILS

The proposed system implements a content-based music similarity and recommendation engine using low-level and mid-level audio descriptors extracted directly from raw audio signals.

The architecture consists of two tightly coupled components:

- Feature extraction and similarity modeling engine implemented in Python using librosa, NumPy, and scikit-learn.
- Interactive visualization and recommendation interface built with Streamlit and Plotly.

Unlike collaborative filtering approaches, the system operates solely on acoustic characteristics, allowing similarity estimation even in the absence of user interaction data.

A. Pipeline

1) Load audio files

```
self.song_files = list(self.audio_dir.glob("*.mp3"))
self.song_names = [f.stem for f in self.song_files]
```

What happens:

-Read all audio files from the folder.

-Store the song names for reference.

-Each audio file is loaded as a mono time-domain signal using librosa.load:

$$x(t) \in \mathbb{R}^N$$

2) Extract features from each audio file

Function: `_extract_features(audio_path)`

a) Tempo estimation

```
tempo_result = librosa.beat.beat_track(y=y, sr=sr)
tempo = float(tempo_result[0])
```

Computes beats per minute (BPM) to global rhythmic descriptor.

b) Spectral centroid

```
spectral_centroid = np.mean(librosa.feature.spectral_centroid(y=y, sr=sr))
```

Measures the “center of mass” of the frequency spectrum and serves as a proxy for perceived energy and brightness.

c) Zero-crossing rate (ZCR)

```
zero_crossing_rate=np.mean(librosa.feature.zero_crossing_rate(y))
```

Quantifies how frequently the audio waveform crosses the zero amplitude level and is used as an approximation of danceability and rhythmic density.

d) Root Mean Square Energy (RMS)

```
rmse = np.mean(librosa.feature.rms(y=y))
```

Represents the average signal loudness and overall intensity of the audio.

e) MFCCs

```
mfccs = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=13)
for i in range(13):
    mfcc_mean = np.mean(mfccs[i])
    mfcc_std = np.std(mfccs[i])
```

Capture timbre, harmonic content, texture.

f) Construct feature vector

```
fi =[tempo, C, ZCR, RMS, μ1, σ1,..., μ13 , σ13 ]
```

3) Aggregate features for all songs

```
features_list = []
for file in self.song_files:
    features = _extract_features(file)
    features_list.append(features)
self.features_df = pd.DataFrame(features_list)
```

Missing values are handled through mean imputation, where NaN values are replaced by the average of the corresponding feature column.

Result: DataFrame of size $N \times 30$.

4) Normalize features

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Standardize each feature to mean 0, std 1:

$$f'_i = \frac{f_i - \mu}{\sigma}$$

5) Compute pairwise distances

```
distances = euclidean_distances(X_scaled)
```

Compute Euclidean distance between each pair of songs:

$$d(i, j) = \sqrt{\sum_{k=1}^D (f'_{i,k} - f'_{j,k})^2}$$

6) Convert distances to similarity scores

```
similarities = 1 / (1 + distances)
self.similarity_matrix = similarities
```

Converts pairwise distances into similarity scores bounded within the interval

$$s(i, j) \in (0, 1]$$

Produces a symmetric similarity matrix of size $N \times N$

7) Apply user preference adjustments

```
if adjustments:
    # energy_adj, dance_adj, tempo_adj
    similarities += 0.1 * (alpha * energy + beta * dance +
                           gamma * tempo)
```

Normalize features for adjustment:

$$x_{\text{norm}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Adjust similarity scores with small weighting factor (0.1) to prevent extreme changes.

8) Recommend top-N similar songs

```
sorted_indices = np.argsort(similarities[idx])[:, :-1]
top_indices = [i for i in sorted_indices if i !=
                idx][:n]
recommendations = {self.song_names[i]:
                    similarities[idx][i] for i in top_indices}
```

Sort similarity scores in descending order
Exclude the query song
Return top-N most similar songs

9) Visualize features (Musical DNA radar chart)

```
features = finder.get_features(selected_song)
radar_values = normalized(mapped_features)
```

Normalize features to [0,1]

Display as radar chart: tempo, energy, danceability, brightness, complexity
Optional audio playback in Streamlit

V. RESULTS

The designed content-based music similarity system is successful in grouping acoustically similar songs using audio features derived directly from the audio files.

Visual observation of the generated recommendation results reveals that songs rated most similar to a particular song possess similar temporal, timbral, and overall energy patterns.

This confirms that the chosen feature vector captures meaningful musical cues, which correlate with human comprehensions of song similarity.

Compared to the baseline strategy that utilizes only the MFCC features with cosine similarity, the proposed strategy provides more coherent suggestions, especially in rhythm-based genres such as electronic/electropop songs.

Incorporating more features such as tempo, spectral centroid, zero-crossing rate, and RMS, which are related to rhythm and intensity, has helped the system to be more sensitive to rhythm.

Unlike the collaborative filtering algorithms, the proposed system does not require user interaction data and is therefore robust to the cold-start problem.

While collaborative methods may achieve higher personalization when extensive user histories are available, the content-based approach provides reliable similarity estimation solely from audio content, making it suitable for small or newly created music collections.

Overall, the results demonstrate that combining multiple acoustic descriptors with feature normalization and Euclidean distance-based similarity yields more musically consistent recommendations than single-feature or MFCC-only baselines, while maintaining simplicity and computational efficiency.

VI. CONCLUSION

This paper presented Music Similarity Finder, a content-based music similarity and music recommendation tool that utilizes low-level and mid-level music descriptors derived from the raw music signal.

The tool combines tempo, spectral features, energy measures, and MFCC-based timbral representations into a unified feature vector, which is normalized and compared using Euclidean distance to estimate musical similarity.

The tool also has an interactive component where users can personalize music recommendations based on preferences for music energy, danceability, and tempo.

Music Similarity Finder adopts a transparent, Euclidean-based model for simplicity and interpretability, while leaving room for more advanced embeddings in future work.

Future extensions could include:

- The integration of song mood analysis and recommendation of songs that have similar moods,
- Multi-modal integration combining audio features with lyrical analysis,
- The integration of learned audio embeddings from deep neural networks,
- Hybrid approaches that combine content-based and collaborative filtering methods

REFERENCES

- [1] G. Tzanetakis and P. Cook, "Musical genre classification of audio signals," *IEEE Transactions on Speech and Audio Processing*, vol. 10, no. 5, pp. 293–302, 2002.
- [2] S. Davis and P. Mermelstein, "Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences," *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 28, no. 4, pp. 357–366, 1980.
- [3] B. Logan, "Mel frequency cepstral coefficients for music modelling," in *Proceedings of the International Symposium on Music Information Retrieval (ISMIR)*, 2000.
- [4] B. McFee, C. Raffel, D. Liang, D. P. W. Ellis, M. McVicar, E. Battenberg, and O. Nieto, "librosa: Audio and music signal analysis in Python," in *Proceedings of the 14th Python in Science Conference*, pp. 18–25, 2015.
- [5] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [6] E. Pampalk, "Computational models of music similarity and their application in music information retrieval," Ph.D. dissertation, Vienna University of Technology, 2006.
- [7] J. J. Rocchio, "Relevance feedback in information retrieval," in *The SMART Retrieval System: Experiments in Automatic Document Processing*, G. Salton, Ed. Englewood Cliffs, NJ, USA: Prentice-Hall, 1971, pp. 313–323.
- [8] A. Treuille et al., "Streamlit: A framework for building data applications," 2019. [Online]. Available: <https://streamlit.io>
- [9] Plotly Technologies Inc., "Plotly: Collaborative data science," 2015. [Online]. Available: <https://plotly.com>